

MAANG System Design Mentorship Track

Goal: build topic-by-topic system design mastery with architect-level depth, interview-ready communication, and practical code intuition.

How We Will Use This Document

- We will append every new topic to this same file.
- Each topic will follow the same structure so your thinking becomes repeatable in interviews.
- We will include code samples and small programs where they help explain the mechanism.
- We will optimize for three outcomes: understanding, recall, and interview communication.

Topic Template

Copy this section for every new topic and fill it in with the topic-specific content.

```
# Topic N: <Topic Name>

---

## 1. Intuition

Explain the concept using a simple analogy or mental model.

Questions to answer:
- What is the simplest way to "feel" this concept?
- If I had to explain it to a beginner in 2-3 lines, how would I do it?

---

## 2. Definition

Give a crisp technical definition in 1-3 lines.

Template:
- Definition:
- Category:
- Core idea:

---

## 3. Why It Exists

Explain why this concept was created.

Questions to answer:
- What problem does it solve?
```

- Why are naive or simpler approaches not enough?
- What breaks without this concept?

4. Reality

Connect the concept to real systems.

Questions to answer:

- Where is it used?
- Which systems or products rely on it?
- What kind of teams or architectures use it often?

5. How It Works

Describe the flow step by step.

Suggested format:

1. Step 1
2. Step 2
3. Step 3
4. Failure path
5. Recovery path

Include:

- Control flow
- Data flow
- Important states
- Failure handling

6. What Problem It Solves

State the exact class of problems it addresses.

Template:

- Primary problem solved:
- Secondary benefits:
- Systems impact:

7. When to Rely on It

Describe when this is the right choice.

Questions to answer:

- In what system conditions is this a strong fit?
- What constraints make it valuable?
- What interviewer keywords should trigger this concept?

8. When Not to Use It

Architect-level maturity comes from knowing when to avoid a tool.

Questions to answer:

- When is it overkill?
- When does it harm performance, cost, or availability?
- What should we use instead?

9. Pros and Cons

Pros	Cons
Pro 1	Con 1
Pro 2	Con 2
Pro 3	Con 3

10. Trade-offs and Common Mistakes

Split this into two parts.

Trade-offs

- What do we gain?
- What do we give up?
- How does it affect latency, throughput, consistency, cost, and complexity?

Common Mistakes

- Mistake:
- Why it is wrong:
- Better approach:

11. Key Numbers

Capture the numbers interviewers expect you to reason with.

Examples:

- Typical latency:
- Throughput:
- Replication factor:
- TTL:
- Partition count:
- Failure threshold:
- Storage growth:

Note:

- Use approximate ranges when exact values vary by system.

12. Failure Modes

Show how the design behaves under stress and partial failure.

Questions to answer:

- What can fail?
- What does the user observe?
- How does the system recover?
- What fallback or mitigation exists?

13. Scenario

Give one real-world system design use case.

Template:

- Product / system:
- Why this concept fits:
- What would go wrong without it:

14. Code Sample

Add a small, focused code snippet that demonstrates the mechanism.

Suggested examples:

- Java
- Python
- SQL
- Pseudocode

```
```java
// Example placeholder
public class Example {
 public static void main(String[] args) {
 System.out.println("Replace with topic-specific sample");
 }
}
```
```

15. Mini Program / Simulation

Add a slightly bigger runnable example when it helps make the concept concrete.

Good candidates:

- Cache simulation
- Load balancer routing
- Rate limiter implementation
- Queue consumer / producer flow
- Consistent hashing demo
- Retry with backoff

```
```python
def main():
 print("Replace with topic-specific simulation")

if __name__ == "__main__":
 main()
```

```

16. Practical Question

Write a realistic interview-style question.

Template:

> You are designing <system>. How would you use <topic> and what trade-offs would you consider?

17. Strong Answer

Write the answer in a crisp, interviewer-friendly structure.

Suggested structure:

1. State whether you would use it.
2. Explain why.
3. Describe how it fits into the design.
4. Mention trade-offs.
5. Mention an alternative.
6. Mention failure handling.

18. Revision Notes

Keep a short summary for fast recall.

Template:

- One-line summary:
- Three keywords:
- One interview trap:
- One memory trick:

Interview Answer Pattern

For any system design answer, try to speak in this order:

1. Clarify the requirement.
2. State the main constraint.
3. Choose the pattern or component.
4. Explain why it fits.
5. Call out trade-offs.
6. Mention failure handling.
7. Mention scale numbers.
8. Offer an alternative if assumptions change.

First Few Suggested Topics

We can start with any order you prefer, but this sequence builds strong fundamentals:

1. Load Balancer
 2. Caching
 3. Database Indexing
 4. Replication
 5. Partitioning / Sharding
 6. Consistent Hashing
 7. CDN
 8. Message Queue
 9. Rate Limiting
 10. API Gateway
 11. Circuit Breaker
 12. Idempotency
 13. 2PC
 14. Saga Pattern
 15. CAP Theorem
-

Working Agreement

- We will do one topic at a time.
 - For each topic, we will fill this template completely.
 - I will keep the explanation at MAANG interview depth, but still intuitive.
 - Where useful, I will add code, mini programs, and interviewer-style Q&A.
-

Topic 1: Request-Response Communication Model

Track: Communication Models & Real-Time Systems Scope: REST APIs, gRPC, GraphQL, Idempotency, Pagination, Versioning

1. Intuition

Think of request-response like calling a service desk.

You ask a clear question, wait while the system checks records or performs an action, and then get a direct answer back. The user experience feels "real-time" because the caller is blocked waiting for a bounded reply.

The three most common styles are:

- REST: "Here is the resource and the action over HTTP."
- gRPC: "Here is the strongly typed remote procedure."
- GraphQL: "Here are exactly the fields I want."

Then production concerns make the model safe and scalable:

- Idempotency prevents duplicate side effects during retries.
 - Pagination prevents huge list responses.
 - Versioning lets old and new clients coexist.
-

2. Definition

- Definition: Request-response is a synchronous communication model in which a client sends a request to a server and expects a bounded reply.
 - Category: Client-server, synchronous communication pattern
 - Core idea: The caller wants an immediate result, acknowledgment, or failure instead of eventual processing.
-

3. Why It Exists

Systems need a simple way to serve user actions and reads where the client is actively waiting.

It exists because:

- Users need immediate feedback for actions like login, search, checkout, and profile updates.
- Services often need fast synchronous coordination for short-lived operations.
- Clients need explicit success, failure, status code, and payload semantics.

Without request-response:

- Every interaction would need asynchronous orchestration, polling, or callbacks.
 - Simple operations would become harder to reason about.
 - User-facing flows would feel indirect and delayed.
-

4. Reality

This pattern is everywhere, but the style depends on the boundary and the workload.

REST

Used most often for:

- Public APIs
- Web and mobile backends
- CRUD-heavy business systems
- Internet-facing services where debuggability and wide compatibility matter

Why teams like it:

- Human-readable JSON
- Easy to test with curl/Postman
- Works naturally with HTTP semantics, proxies, caching, and gateways

gRPC

Used most often for:

- Service-to-service communication inside microservice platforms
- Low-latency internal RPCs
- Strongly typed contracts
- Polyglot backends that need generated clients

Why teams like it:

- Compact binary payloads
- HTTP/2 multiplexing
- Strong schema contracts with Protocol Buffers

GraphQL

Used most often for:

- Client-specific aggregation layers
- Mobile apps with multiple screen shapes
- Frontend-heavy products where over-fetching is painful
- Backend-for-frontend (BFF) layers

Why teams like it:

- Clients ask for only the fields they need
- Many resource fetches can be unified behind a single endpoint
- Schema can evolve additively

Cross-cutting concerns

- Idempotency is critical for payments, order creation, retries, and any side-effecting operation.
 - Pagination is necessary for feeds, search, admin lists, audit logs, and time-series views.
 - Versioning matters when clients upgrade slowly or contracts must evolve safely.
-

5. HOW IT WORKS

Generic request-response flow

1. A client sends a request with a contract: method, path or RPC name, headers, auth, parameters, and body.
2. A gateway, load balancer, or service endpoint receives the request.
3. The server authenticates, authorizes, validates, and rate-limits.
4. Business logic executes: read cache, query database, call downstream services, or perform a mutation.
5. The server returns a response: status code, payload, metadata, and possibly pagination cursor or version headers.
6. The client either renders the response or retries on failure.

REST flow

1. Client calls an HTTP endpoint such as `GET /v1/products/123`.
2. Server maps the route to resource-oriented logic.
3. Response typically returns JSON plus an HTTP status code.

gRPC flow

1. Client calls a generated method such as `ReserveInventory()`.
2. Request and response are serialized using Protocol Buffers.
3. HTTP/2 carries the RPC efficiently between services.

GraphQL flow

1. Client posts a query to a GraphQL endpoint.
2. The server validates the query against the schema.
3. Resolvers fetch only the requested fields.
4. The server returns a response shaped like the query.

Idempotency during retries

1. Client sends a write request with an `Idempotency-Key`.
2. Server checks whether that key was already processed.
3. If yes, it returns the same logical result.
4. If no, it executes once and stores the result against the key.

Pagination

1. Client requests a limited subset of data.
2. Server returns items plus pagination metadata.
3. Client requests the next page using offset or cursor.

Versioning

1. Server exposes a contract such as `/v1/orders`.
 2. Additive changes are handled without breaking old clients when possible.
 3. Breaking changes move to a new version such as `/v2/orders`.
-

6. What Problem It Solves

- Primary problem solved: Safe, bounded, synchronous interaction between clients and services
 - Secondary benefits: Clear contracts, easy debugging, direct error reporting, cache friendliness, simple mental model
 - Systems impact: Shapes API design, latency budgets, client coupling, retry behavior, and schema evolution strategy
-

7. When to Rely on It

Use request-response when:

- The caller is actively waiting for an answer.
- The operation should complete within a predictable latency budget.
- The client needs immediate success/failure semantics.
- The interaction is read-heavy or short mutation-heavy.

Use REST when:

- The API is public or browser/mobile facing.
- Simplicity and ecosystem compatibility matter more than raw efficiency.
- Resource-oriented modeling is natural.

Use gRPC when:

- Calls are mostly internal between services.
- Low latency and compact payloads matter.
- You want strict schemas and generated clients.

Use GraphQL when:

- Different clients need different data shapes.
- One screen requires data from many backends.
- You want to reduce over-fetching and round trips at the client layer.

Use idempotency when:

- A write request may be retried.
- Double execution is costly or dangerous.
- External networks, clients, or payment flows can resend requests.

See cursor pagination when:

- Data is large, frequently changing, or ordered by time/id.

Use versioning when:

- Multiple client versions must coexist.
 - Contract changes may break old consumers.
-

8. When Not to Use It

Avoid pure request-response when:

- The job is long-running, such as video processing or report generation.
- The workload is naturally asynchronous and decoupled.
- The server needs to push updates continuously.
- The system is event-driven and loose coupling matters more than immediate reply.

Better alternatives:

- Queue + worker for long-running jobs
- Event-driven architecture for decoupled services
- WebSockets or SSE for server push and live updates
- Streaming systems for telemetry and continuous pipelines

Avoid GraphQL when:

- The use case is simple CRUD and operational simplicity matters most.
- Teams are not prepared to handle resolver performance and query governance.

Avoid gRPC at the public internet edge when:

- Browser compatibility, proxy friendliness, and human debuggability are top priorities.

Avoid offset pagination when:

- The dataset changes frequently and consistency of browsing matters.
-

9. Pros and Cons

| Pros | Cons |
|---|--|
| Simple mental model with direct request -> response semantics | Tight temporal coupling: caller waits for callee |
| Easy to expose clear errors, statuses, and contracts | Tail latency propagates across service chains |
| Works well for user-facing reads and short writes | Retries can duplicate side effects without idempotency |

| | |
|--|--|
| REST is easy to debug and widely supported | REST can over-fetch or under-fetch for complex screens |
| gRPC is efficient and strongly typed | gRPC is less browser-friendly and harder to inspect manually |
| GraphQL gives clients flexible data shapes | GraphQL can create N+1 queries and expensive resolver graphs |

10. Trade-offs and Common Mistakes

Trade-offs

- REST vs gRPC: REST is simpler and more universal; gRPC is faster and more structured for internal calls.
- REST vs GraphQL: REST is operationally simpler; GraphQL gives clients more flexibility but increases server-side complexity.
- Offset vs cursor pagination: Offset is easy to implement; cursor is more stable and scalable on changing datasets.
- Idempotency safety vs storage/state: Idempotency reduces duplicate writes but requires storing keys and previous outcomes.
- Versioning stability vs maintenance cost: Versioning protects old clients but increases API surface area and migration effort.

Common Mistakes

| Mistake | Why it is wrong | Better approach |
|---|---|--|
| Using synchronous request-response for long-running work | Causes client timeouts and wasted retries | Return <code>202 Accepted</code> , enqueue work, expose job status |
| Missing idempotency on <code>__CODE0</code> <i>or</i> <code>CODE1</code> __ | Retries can create duplicate charges or orders | Require <code>Idempotency-Key</code> and store execution result |
| Using offset pagination on hot feeds | Inserts/deletes cause skipped or duplicated items | Use cursor pagination based on stable ordering |
| Breaking response schemas without versioning | Old clients fail unexpectedly | Prefer additive changes, then introduce <code>v2</code> for breaking changes |
| Choosing GraphQL without resolver batching | Leads to N+1 query explosions | Use batching, caching, and query limits |
| Building deep chatty synchronous service chains | Increases p95/p99 latency and failure propagation | Collapse hops, cache, or shift to events where appropriate |

11. Key Numbers

These are interview heuristics, not hard laws.

- Public API read latency target: often p95 under 100-300 ms for common user actions
 - Internal same-region service call: often p50 around 5-20 ms, p95 around 20-100 ms depending on work
 - End-to-end timeout budget: often 200 ms to 2 s depending on the product path
 - Retry count: usually 1-3 attempts max with exponential backoff and jitter
 - Idempotency key retention: commonly 24-72 hours for payment/order style flows
 - Default page size: often 20-100 items for user-facing APIs
 - Internal bulk page size: sometimes 100-1000 items if memory and latency permit
 - GraphQL query depth/complexity: usually guarded with server-side limits
 - Offset pagination pain point: large offsets degrade because the database still scans/skips
 - Versioning rule of thumb: additive changes first, new version only for true contract breaks
-

12. Failure Modes

Client timeout after server success

Problem:

- The server completed the mutation, but the client timed out and retries.

User impact:

- Duplicate orders, duplicate payments, or duplicate reservations if writes are not idempotent.

Mitigation:

- Idempotency keys
- Request tracing
- Store and replay the same logical response

Downstream latency spike

Problem:

- One dependency gets slow, and the synchronous chain inherits the delay.

User impact:

- High p95/p99 latency or failed requests.

Mitigation:

- Tight timeouts
- Circuit breakers
- Fallbacks

Breaking API changes

Problem:

- New server response no longer matches old client expectations.

User impact:

- Mobile/web clients fail after backend rollout.

Mitigation:

- Backward-compatible additive evolution
- Explicit versioning
- Consumer contract testing

GraphQL N+1 resolver explosion

Problem:

- A single query triggers many database calls.

User impact:

- Slow pages and backend overload.

Mitigation:

- Resolver batching
- DataLoader-style batching
- Query complexity guards

Pagination drift

Problem:

- New rows inserted between page requests change the result ordering.

User impact:

- Missing or duplicate items while paging.

Mitigation:

- Cursor-based pagination with stable sort keys

13. Scenario

- Product / system: E-commerce platform with web, mobile, and internal microservices
- Why this concept fits: Users need immediate product reads, cart updates, order placement, and order status lookups.
- Recommended design: Public APIs use REST for broad compatibility, the web BFF may expose GraphQL for homepage/product detail aggregation, and internal services use gRPC for low-latency typed calls.

~~Production hardening: __CURSOR__ idempotency key, pagination, and breaking contract changes move from CODE1 to CODE_2__.~~

- What would go wrong without it: Clients would over-fetch data, duplicate writes would appear during retries, large list APIs would become slow, and schema changes would break old clients.

14. Code Sample

REST examples

```
GET /v1/products?limit=20&cursor=eyJzYXN0SWQiOjEyM30=
Authorization: Bearer <token>
```

```
POST /v1/orders
Idempotency-Key: 3b0d5ad1-6e6d-4c63-96f3-3fc6ef4f92b4
Content-Type: application/json

{
  "userId": "u-101",
  "items": [
    { "sku": "iphone-256", "quantity": 1 }
  ],
  "paymentMethodId": "pm-44"
}
```

gRPC contract example

```
syntax = "proto3";

service InventoryService {
  rpc ReserveInventory(ReserveInventoryRequest)
    returns (ReserveInventoryResponse);
}

message ReserveInventoryRequest {
  string order_id = 1;
  string sku = 2;
  int32 quantity = 3;
}

message ReserveInventoryResponse {
  bool reserved = 1;
  string reservation_id = 2;
}
```

GraphQL example

```
query ProductPage($productId: ID!, $cursor: String) {
  product(id: $productId) {
    id
    name
  }
}
```

```

    price
    reviews(first: 5, after: $cursor) {
      edges {
        node {
          rating
          comment
        }
      }
    }
    pageInfo {
      endCursor
      hasNextPage
    }
  }
}

```

Java Spring Boot idempotent create endpoint

```

@RestController
@RequestMapping("/v1/orders")
public class OrderController {

    private final IdempotencyService idempotencyService;
    private final OrderService orderService;

    public OrderController(IdempotencyService idempotencyService, OrderService
orderService) {
        this.idempotencyService = idempotencyService;
        this.orderService = orderService;
    }

    @PostMapping
    public ResponseEntity<OrderResponse> createOrder(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @RequestBody CreateOrderRequest request) {

        OrderResponse response = idempotencyService.executeOnce(
            "create-order",
            idempotencyKey,
            () -> orderService.createOrder(request)
        );

        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }
}

```

Key idea:

- If the client retries the same request with the same idempotency key, the server returns the same logical result instead of creating another order.

15. Mini Program / Simulation

This mini program shows two production ideas together:

- cursor pagination over a growing order list

```
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

@dataclass
class Order:
    order_id: int
    user_id: str
    amount: int

class OrderApi:
    def __init__(self) -> None:
        self.next_id = 1
        self.orders: List[Order] = []
        self.idempotency_store: Dict[str, Order] = {}

    def create_order(self, idempotency_key: str, user_id: str, amount: int) -> Order:
        if idempotency_key in self.idempotency_store:
            return self.idempotency_store[idempotency_key]

        order = Order(order_id=self.next_id, user_id=user_id, amount=amount)
        self.next_id += 1
        self.orders.append(order)
        self.idempotency_store[idempotency_key] = order
        return order

    def list_orders(self, limit: int, cursor: Optional[int] = None) -> Tuple[List[Order],
Optional[int]]:
        start_index = 0

        if cursor is not None:
            for i, order in enumerate(self.orders):
                if order.order_id > cursor:
                    start_index = i
                    break
            else:
                return [], None

        page = self.orders[start_index:start_index + limit]
        next_cursor = page[-1].order_id if len(page) == limit else None
        return page, next_cursor

def main() -> None:
    api = OrderApi()

    first = api.create_order("req-1001", "user-1", 5000)
    retry = api.create_order("req-1001", "user-1", 5000)
    second = api.create_order("req-1002", "user-1", 2500)
    third = api.create_order("req-1003", "user-2", 1800)

    print("First order:", first)
    print("Retry with same key returns same order:", retry)
```

```

print("Created more orders:", second, third)

page_1, cursor_1 = api.list_orders(limit=2)
print("Page 1:", page_1, "next_cursor:", cursor_1)

page_2, cursor_2 = api.list_orders(limit=2, cursor=cursor_1)
print("Page 2:", page_2, "next_cursor:", cursor_2)

if __name__ == "__main__":
    main()

```

What this demonstrates:

- Same key -> same side effect only once
- Cursor points to the last seen item
- Next page starts after the cursor, not from a fragile offset

16. Practical Question

You are designing APIs for a large online marketplace. Mobile and web clients need product details, search results, cart operations, and checkout. Internal services need fast synchronous communication for inventory, pricing, and order validation. How would you use REST, gRPC, GraphQL, idempotency, pagination, and versioning in this design?

17. Strong Answer

I would use a mixed request-response model based on the boundary.

For public APIs, I would default to REST because it is simple, debuggable, and broadly compatible with browsers, mobile apps, gateways, and CDN-friendly HTTP tooling. For client-specific aggregation, especially homepage or product detail screens that need data from multiple services, I would expose GraphQL through a BFF layer so the client can fetch exactly the fields it needs without multiple round trips. For internal microservice-to-microservice synchronous calls, I would prefer gRPC because it gives us strongly typed contracts, smaller payloads, and lower overhead.

For safety, I would make all critical write endpoints such as order creation and payment authorization idempotent using an `__CODE0`, **because retries are unavoidable in distributed systems and we cannot risk duplicate orders or charges. For list endpoints like search results, orders, or activity feeds, I would use cursor pagination instead of offset pagination once data becomes large or frequently changing. For API evolution, I would keep changes backward compatible when possible and introduce `__CODE1` only for true breaking changes.**

The main trade-off is operational simplicity versus flexibility. REST is simplest, gRPC is fastest internally, and GraphQL is most flexible for clients but needs resolver batching and query governance. If an operation becomes long-running, I would stop using pure request-response and switch to ²⁰² Accepted plus asynchronous processing.

18. Revision Notes

- One-line summary: Request-response is the default model for bounded, synchronous interactions, and production API quality comes from choosing the right style plus adding idempotency, pagination, and versioning.
 - Three keywords: synchronous, contract, retries
 - One interview trap: treating REST, gRPC, and GraphQL as mutually exclusive instead of using each at the right boundary
 - One memory trick: REST for reach, gRPC for speed, GraphQL for shape
-

Deep Dive: Pagination in Request-Response APIs

1. What pagination means at the API level

Pagination is not just a database trick. At the request-response level, it is an API contract for returning large collections in smaller, ordered chunks.

The client says:

- "Give me the first N items"
- "Give me the next N items after this point"
- or "Give me page 3"

The server replies with:

- the current slice of items
- enough metadata to fetch the next slice

This matters even if the backend data is not coming directly from a database. The data could come from:

- a database
- a search engine
- a cache
- a downstream service
- a composed GraphQL resolver

So think of pagination as:

- API-level contract first
 - datastore query strategy second
-

2. How pagination works in a request-response flow

1. Client requests a bounded list.
2. Server applies filters, sorting, and page boundaries.
3. Server fetches only the required slice from the backend.

4. Server returns items plus paging metadata.

5. Client uses that metadata to request the next slice.

The most important design rule:

- Pagination only works correctly when ordering is deterministic.

That means the server must define a stable sort such as:

- `ORDER BY created_at DESC, id DESC`

Using only `__CODE0` **is unsafe when many rows share the same timestamp. Add a tie-breaker like `CODE1`**.

3. Common API pagination styles

There are really three common request-response styles:

1. Page-number pagination
2. Offset-limit pagination
3. Cursor-based pagination

Page-number pagination and offset-limit pagination are closely related. Page number is mostly a client-friendly wrapper over offset.

4. Page-number pagination

Example request:

```
GET /v1/products?page=3&pageSize=20
```

What the server does:

- Computes `offset = (page - 1) * pageSize`
- Runs a query like `LIMIT 20 OFFSET 40`

Example response:

```
{
  "items": [
    { "id": 41, "name": "Keyboard" },
    { "id": 42, "name": "Mouse" }
  ],
  "page": 3,
  "pageSize": 20,
  "totalCount": 1200,
  "totalPages": 60
}
```

Why teams use it:

- Very easy for UI tables with numbered pages
- Easy to explain

Weakness:

- It inherits all offset problems under the hood

5. Offset-limit pagination

Example request:

```
GET /v1/products?offset=40&limit=20
```

Typical SQL:

```
SELECT id, name
FROM products
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 40;
```

Example response:

```
{
  "items": [
    { "id": 41, "name": "Keyboard" },
    { "id": 42, "name": "Mouse" }
  ],
  "offset": 40,
  "limit": 20,
  "nextOffset": 60
}
```

How it works:

- `offset` tells the server how many matching rows to skip
- `limit` tells the server how many rows to return

Pros:

- Very simple
- Easy to jump to arbitrary pages
- Good for small or mostly static datasets

Cons:

- Large offsets get slower because the backend still has to scan/skip rows
- Inserts/deletes between requests can cause duplicates or missing items
- Not ideal for infinite scroll, feeds, or live data

Good fit:

- Admin tables
- Backoffice dashboards

- Reporting UIs where numbered pages matter

6. Cursor-based pagination

Example request:

```
GET /v1/products?
limit=20&cursor=eyJjcmVhdGVkQXQiOiIyMDI2LTA0LTAyVDEwOjE1OjAwWiIsImlkIjoianNDIifQ==
```

What the cursor means:

- "Give me the next 20 records after the last item I already saw."

The cursor is usually an opaque token that encodes the last seen sort key, for example:

- `created_at`
- `id`
- or both

Typical SQL for keyset/cursor pagination:

```
SELECT id, name, created_at
FROM products
WHERE (created_at, id) < ('2026-04-02T10:15:00Z', 42)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Example response:

```
{
  "items": [
    { "id": 41, "name": "Keyboard", "createdAt": "2026-04-02T10:14:00Z" },
    { "id": 40, "name": "Mouse", "createdAt": "2026-04-02T10:13:00Z" }
  ],
  "nextCursor": "eyJjcmVhdGVkQXQiOiIyMDI2LTA0LTAyVDEwOjE1OjAwWiIsImlkIjoianNDIifQ==",
  "hasMore": true
}
```

How it works:

- The first request usually omits the cursor.
- The server returns the first page plus a `nextCursor`.
- The client sends that cursor in the next request.
- The server continues from the last seen position instead of skipping rows.

Pros:

- Stable for changing datasets
- Better performance on large ordered datasets
- Great for feeds, timelines, transaction history, search results, and infinite scroll

Cursor

- Harder to implement
- Hard to jump directly to "page 37"
- Requires stable sort keys and careful cursor design

Good fit:

- Social feed
- Order history
- Notification timeline
- Search results
- Event logs

7. Offset vs cursor: the real difference

Offset asks:

- "Skip N rows, then give me the next M."

Cursor asks:

- "Start after this exact last seen record, then give me the next M."

That is why cursor pagination behaves better when data changes between requests.

Example:

- You fetch page 1 of a news feed.
- Before you fetch page 2, 5 new posts arrive at the top.

With offset:

- Page 2 may now overlap with items you already saw or skip items you should have seen.

With cursor:

- The server continues after your last seen item, so pagination stays stable.

8. Same idea in REST, GraphQL, and gRPC

REST

Offset style:

```
GET /v1/orders?offset=0&limit=20
```

Cursor style:

```
GET /v1/orders?limit=20&cursor=abc123
```

GraphQL

GraphQL commonly uses the connection model

```
query Orders($first: Int!, $after: String) {  
  orders(first: $first, after: $after) {  
    edges {  
      node {  
        id  
        total  
      }  
    }  
  }  
  pageInfo {  
    endCursor  
    hasNextPage  
  }  
}
```

gRPC

gRPC often uses page tokens:

```
message ListOrdersRequest {  
  int32 page_size = 1;  
  string page_token = 2;  
}  
  
message ListOrdersResponse {  
  repeated Order orders = 1;  
  string next_page_token = 2;  
}
```

Different protocols, same pattern:

- request asks for a bounded slice
- response returns results plus continuation metadata

9. What interviewers want to hear

If an interviewer asks, "How would you paginate this API?", they usually want these decisions:

1. What is the sort order?
2. Is the dataset static or frequently changing?
3. Do users need page numbers or infinite scroll?
4. Do you need stable results under concurrent inserts/deletes?
5. Is total count required, approximate, or optional?

Strong answer pattern:

- "For admin/reporting UIs, I would start with offset/page-number pagination because users often want page jumps."
- "For feeds, search, order history, or other changing datasets, I would prefer cursor pagination because it is more stable and scales better."

10. Important implementation rules

- Always define a deterministic order.
 - Add a tie-breaker such as `id`.
 - Keep cursors opaque; do not let clients depend on cursor internals.
 - Enforce a max `limit` to protect latency and memory.
 - Keep filters and sort order consistent across page requests.
 - If filters change, treat it like a new query and issue a new cursor.
 - Be careful with `totalCount`; it can be expensive on large datasets.
 - For hot paths, prefer returning `__CODE0` **and** `CODE1` instead of exact totals.
-

11. When to choose which

Choose page-number or offset when:

- Users need page 1, page 2, page 3 style navigation
- Data is relatively stable
- Simplicity matters more than perfect consistency

Choose cursor when:

- Data changes frequently
 - You care about stable pagination
 - The list is large
 - You are building feed-like or timeline-like UX
 - You want better performance at scale
-

12. Fast mental model

- Offset = position by count
- Cursor = position by last seen record

Or even shorter:

- Offset is easy
 - Cursor is stable
-

Topic 2: Client-Pull Models

Track: Communication Models & Real-Time Systems Scope: Fixed-interval polling, long polling

1. Intuition

Imagine you are waiting for a parcel update.

With fixed-interval polling, you keep opening the delivery app every 30 seconds and asking, "Any update now?"

With long polling, you ask once and keep the call open: "Please wait and tell me as soon as something changes, otherwise I will ask again after you reply."

Both are still client-pull models because the client initiates every request. The difference is how wasteful or efficient that waiting becomes.

2. Definition

- Definition: Client-pull models are communication patterns where the client repeatedly initiates requests to check for new state or events.
 - Category: Near-real-time request-response communication
 - Core idea: The server does not proactively push unless the client first opens or renews the request.
-

3. Why It Exists

Not every system can use WebSockets, SSE, or message subscriptions.

These models exist because:

- Many networks, browsers, load balancers, and enterprise environments already support plain HTTP well.
- Teams sometimes need near-real-time updates without introducing a dedicated push stack.
- Older systems and simple products often want a lower operational barrier.

Without client-pull:

- Clients would need manual refresh.
 - Users would see stale notifications, job statuses, or chat updates.
 - Systems would jump too early to more complex real-time infrastructure.
-

4. Reality

Client-pull shows up in many practical systems.

Fixed-interval polling is common in:

- Job status pages
- Payment status refresh
- CI/CD pipeline progress screens

- Monitoring panels with refresh every few seconds

Long polling is common in:

- Lightweight chat systems
- Notification systems
- Collaborative tools before full WebSocket adoption
- Event wait APIs
- Systems where "almost push" behavior is needed over HTTP

Why teams still use them

- Simple HTTP semantics
- Easy to support through API gateways and proxies
- Easier to debug than many push protocols
- Good stepping stone before WebSockets or SSE

5. How It Works

Fixed-interval polling flow

1. Client sends a request such as `GET /notifications?since=token-123`.
2. Server immediately responds with current data, which may be empty.
3. Client waits for a configured interval, for example 5 seconds.
4. Client sends the same request again.
5. This repeats continuously.

Key characteristic:

- The client decides the polling interval.

Long polling flow

1. Client sends a request such as `GET /notifications/long-poll?since=token-123`.
2. If data is already available, the server responds immediately.
3. If no data is available, the server keeps the request open for some bounded time, for example 20-30 seconds.
4. If an event arrives during that wait, the server immediately returns the event.
5. If no event arrives before timeout, the server returns an empty response or timeout response.
6. The client immediately opens a new long-poll request.

Key characteristic:

- The client still initiates every request, but the server delays the response until data arrives or timeout occurs.

Fixed polling vs long polling in one line

- Fixed polling = ask every N seconds
 - Long polling = ask once, wait until something happens or timeout, then ask again
-

6. What Problem It Solves

- Primary problem solved: Getting fresh data from the server without manual refresh
 - Secondary benefits: Near-real-time UX over plain HTTP, simpler rollout than full push systems
 - Systems impact: Affects freshness, infrastructure load, connection handling, and perceived responsiveness
-

7. When to Rely on It

Use fixed-interval polling when:

- Updates are infrequent or not highly time-sensitive
- Simplicity matters most
- The UI can tolerate a few seconds of staleness
- The number of clients is moderate

Use long polling when:

- You want near-real-time behavior but cannot use WebSockets or SSE yet
- Updates are irregular
- Immediate delivery after event arrival is helpful
- Empty polling responses would otherwise waste too much traffic

Interview signal words that suggest polling:

- "status page"
 - "job progress"
 - "refresh every few seconds"
 - "can't keep a push connection stack yet"
 - "simple near-real-time updates over HTTP"
-

8. When Not to Use It

Avoid fixed polling when:

- You have millions of active clients
- Updates are rare, making most requests empty
- Freshness requirements are tighter than the polling interval

Why long polling when?

- Very high connection fan-out makes open-request management expensive
- You need full bidirectional communication
- You need sub-second high-frequency updates at large scale

Better alternatives:

- WebSockets for bidirectional live interaction
- SSE for server-to-client event streaming over HTTP
- Pub/sub and event streams for backend distribution
- Webhooks when one server should notify another asynchronously

9. Pros and Cons

| Pros | Cons |
|--|---|
| Polling is easy to implement and reason about | Fixed polling wastes requests when nothing changes |
| Works with existing HTTP infrastructure | Polling introduces staleness between checks |
| Long polling improves freshness over fixed polling | Long polling consumes open connections for longer |
| Good transitional model before full push systems | Both models still keep the client responsible for re-requesting |
| Easy to debug with familiar tools | Large client counts can overload APIs with repeated traffic |

10. Trade-offs and Common Mistakes

Trade-offs

- Simplicity vs freshness: Fixed polling is simple but stale between intervals.
- Freshness vs infrastructure cost: Long polling reduces empty requests but keeps connections open longer.
- HTTP compatibility vs real-time richness: Polling works broadly, but WebSockets provide richer real-time interaction.
- Small scale convenience vs large scale waste: Polling is fine early; at scale it can create a lot of needless load.

Common Mistakes

| Mistake | Why it is wrong | Better approach |
|---------|-----------------|-----------------|
|---------|-----------------|-----------------|

| | | |
|---|--|--|
| Polling too frequently, such as every 500 ms | Creates avoidable load and battery/network waste | Tune interval to business need, cache, or move to push |
| Polling without delta tokens like <code>since</code> or version markers | Sends the full dataset repeatedly | Return only new or changed data |
| Treating long polling like infinite wait | Ties up server resources and proxy timeouts | Use bounded server timeout and reconnect |
| Using polling for highly interactive chat/game presence at scale | Creates poor freshness or massive request volume | Use WebSockets or SSE |
| Ignoring retry jitter after failures | Causes synchronized reconnect storms | Use randomized backoff |

11. Key Numbers

These are practical heuristics and vary by product.

- Fixed polling interval: often 5-30 seconds for dashboards/status pages
- Aggressive near-real-time polling: sometimes 1-5 seconds, but expensive at scale
- Long-poll timeout: often 15-30 seconds, sometimes up to 60 seconds depending on infrastructure
- Reconnect backoff: typically start at 1-2 seconds with jitter
- Empty response rate: if most poll responses are empty, fixed polling is likely inefficient
- Scale warning: $\text{requests per second} \approx \text{active_clients} / \text{polling_interval_seconds}$

Example:

- 100,000 clients polling every 5 seconds = 20,000 requests/sec
- 100,000 clients polling every 30 seconds = about 3,333 requests/sec

That simple math is exactly why interviewers worry about polling.

12. Failure Modes

Thundering herd

Problem:

- Many clients poll at the same time, especially after page load or outage recovery.

User impact:

- API spikes, rate limits, slow responses.

Mitigation:

- Jitter polling intervals
- Stagger reconnects

Empty-response waste

Problem:

- Most poll requests return no new data.

User impact:

- Unnecessary bandwidth, higher infra cost, battery drain on mobile.

Mitigation:

- Increase interval
- Use long polling
- Return deltas instead of full payloads

Timeout chain in long polling

Problem:

- Proxy, load balancer, or application timeouts close requests too early.

User impact:

- Frequent reconnects, missed immediacy, noisy logs.

Mitigation:

- Align timeout settings across client, gateway, and server
- Use bounded wait windows

Reconnect storm

Problem:

- After a deployment or network issue, many clients reconnect together.

User impact:

- Sudden traffic surge and unstable recovery.

Mitigation:

- Exponential backoff with jitter
- Retry budgets

13. Scenario

- Product / system: Video processing dashboard
- Why this concept fits: After a user uploads a video, the UI needs to show states such as `__CODE0`, `CODE1`, `CODE2`, **or** `CODE3`.
- Recommended design: Start with fixed polling every 5-10 seconds for MVP because it is simple and reliable. If users demand faster perceived updates and many responses are empty, move to long polling so the server replies as soon as state changes.

What would go wrong without it? Users would keep manually refreshing and would not trust the system state.

Another strong example:

- Product / system: In-app notifications
- Recommended design: Long polling is a reasonable HTTP-based step before moving to WebSockets or SSE.

14. Code Sample

Fixed polling in browser JavaScript

```
async function pollJobStatus(jobId) {
  const response = await fetch(`/v1/jobs/${jobId}/status`);
  const data = await response.json();

  renderStatus(data);

  if (data.state !== "COMPLETED" && data.state !== "FAILED") {
    setTimeout(() => pollJobStatus(jobId), 5000);
  }
}
```

Long polling in browser JavaScript

```
async function longPollNotifications(lastEventId = "") {
  try {
    const response = await fetch(`/v1/notifications/long-poll?since=${lastEventId}`);
    const data = await response.json();

    if (data.events.length > 0) {
      renderEvents(data.events);
      lastEventId = data.events[data.events.length - 1].id;
    }
  } finally {
    setTimeout(() => longPollNotifications(lastEventId), 0);
  }
}
```

Spring Boot style long-poll endpoint

```
@RestController
@RequestMapping("/v1/notifications")
public class NotificationController {

  private final NotificationService notificationService;

  public NotificationController(NotificationService notificationService) {
    this.notificationService = notificationService;
  }
}
```



```

@GetMapping("/long-poll")
public DeferredResult<ResponseEntity<NotificationResponse>> longPoll(
    @RequestParam(required = false) String since) {

    DeferredResult<ResponseEntity<NotificationResponse>> result =
        new DeferredResult<>(25_000L);

    notificationService.findOrWaitForEvents(since, events -> {
        result.setResult(ResponseEntity.ok(new NotificationResponse(events)));
    });

    result.onTimeout(() ->
        result.setResult(ResponseEntity.ok(new NotificationResponse(List.of()))));

    return result;
}

```

Key idea:

- The thread should not block the whole time in a naive way.
- Use async request handling and bounded timeout.

15. Mini Program / Simulation

This simulation compares request volume for fixed polling and long polling under the same event stream.

```

def simulate_fixed_polling(duration_seconds: int, poll_interval: int) -> int:
    requests = 0
    current = 0

    while current < duration_seconds:
        requests += 1
        current += poll_interval

    return requests

def simulate_long_polling(duration_seconds: int, event_times: list[int], timeout: int) ->
int:
    requests = 0
    current = 0
    index = 0

    while current < duration_seconds:
        requests += 1
        wait_until = min(current + timeout, duration_seconds)

        if index < len(event_times) and current < event_times[index] <= wait_until:
            current = event_times[index]
            index += 1
        else:
            current = wait_until

```

```

return requests

def main() -> None:
    duration = 60
    event_times = [12, 39, 55]

    fixed_requests = simulate_fixed_polling(duration, poll_interval=5)
    long_poll_requests = simulate_long_polling(duration, event_times=event_times,
    timeout=20)

    print("Duration:", duration, "seconds")
    print("Events happened at:", event_times)
    print("Fixed polling requests:", fixed_requests)
    print("Long polling requests:", long_poll_requests)

if __name__ == "__main__":
    main()

```

What this demonstrates:

- Fixed polling keeps asking even when nothing changes
- Long polling usually sends fewer empty requests when events are sparse
- Traffic pattern matters as much as freshness

16. Practical Question

You are designing a job-status and notification system for a SaaS platform. Users upload files, wait for processing, and expect near-real-time updates in the UI. You are not allowed to introduce WebSockets in the first version. Would you use fixed polling or long polling, and how would you scale it?

17. Strong Answer

I would choose based on freshness requirements and traffic shape.

For a job-status page, I would start with fixed-interval polling because it is very simple and usually good enough. The client can request status every 5-10 seconds, and the server can return a compact state such as `__CODE0, CODE1, CODE2, or CODE3__`. This is especially practical for an MVP or for moderate traffic.

For notifications or other irregular event-driven updates, I would prefer long polling over fixed polling because it reduces the number of empty responses and improves perceived real-time behavior. The client sends a request, the server waits up to a bounded timeout for new events, and the client immediately reconnects after each response. I would implement this with async request handling, timeout alignment across load balancer and app server, and reconnect jitter to avoid herd behavior.

The trade-off is that long polling is more efficient than fixed polling when updates are sparse, but it consumes open connections longer. If scale grows further or the product needs richer bi-directional real-

18. Revision Notes

- One-line summary: Polling is the simplest client-pull model, and long polling is the more efficient HTTP-based option when you want near-real-time updates without full push infrastructure.
 - Three keywords: pull, interval, timeout
 - One interview trap: saying long polling is server push; it is still client-initiated pull
 - One memory trick: polling asks often, long polling asks and waits
-

Topic 3: Server-Push Models

Track: Communication Models & Real-Time Systems Scope: WebSockets (full-duplex), Server-Sent Events or SSE (server-to-client push)

1. Intuition

Think of two different real-time conversations.

WebSockets are like a live phone call:

- both sides stay connected
- both sides can speak at any time
- great when the client and server both need to send messages frequently

SSE is like a live news ticker or radio channel:

- the client tunes in once
- the server keeps sending updates
- the client mostly listens

Both avoid repeated request-per-update overhead. The big difference is:

- WebSockets = two-way real-time channel
 - SSE = one-way stream from server to client
-

2. Definition

- Definition: Server-push models keep a long-lived connection or stream so the server can send updates without requiring a fresh client request for each event.
- Category: Real-time communication patterns
- Core idea: Reduce latency and repeated request overhead by keeping the channel open for

3. Why It Exists

Polling works, but it wastes work and adds delay.

Server-push exists because:

- users expect live updates for chat, collaboration, notifications, and streaming dashboards
- repeated polling creates empty requests and infrastructure waste
- some products need the server to react immediately when new events happen

Without server-push:

- chat feels delayed
- live feeds feel stale
- interactive apps either over-poll or under-deliver freshness

4. Reality

WebSockets are common in:

- chat systems
- multiplayer presence and collaboration tools
- trading dashboards
- gaming backends
- ride tracking or live dispatch maps
- operational consoles with user actions flowing both ways

Why teams choose them:

- low-latency bi-directional messaging
- fewer repeated HTTP requests
- natural fit for live interaction

SSE is common in:

- notification feeds
- live scoreboards
- stock tickers
- activity streams
- monitoring dashboards
- AI response streaming and other append-only server output

Why teams choose it:

Simpler than WebSockets

- built on normal HTTP semantics
- easy fit when the browser/client only needs server-to-client updates

Neighboring pattern worth knowing

Webhooks are also push-like, but they are server-to-server callbacks, not browser real-time streams. Mention them only when the receiver is another backend, not an end user UI.

5. How It Works

WebSocket flow

1. Client opens an HTTP request with an `Upgrade: websocket` header.
2. Server accepts the upgrade and the protocol switches from HTTP to WebSocket.
3. A persistent TCP connection stays open.
4. Client and server exchange frames whenever needed.
5. Heartbeats or ping/pong messages keep the connection healthy.
6. On disconnect, the client reconnects and resumes from the latest known state if needed.

Key property:

- full-duplex communication, meaning both sides can send independently.

SSE flow

1. Client makes a standard HTTP request such as `GET /events/stream`.
2. Server responds with `Content-Type: text/event-stream`.
3. The connection stays open.
4. The server pushes events line by line as they happen.
5. The client listens continuously and processes each event.
6. If the connection drops, the client reconnects, often with a last event ID.

Key property:

- unidirectional stream from server to client over standard HTTP.

One-line contrast

- WebSocket = persistent two-way message pipe
 - SSE = persistent one-way event stream
-

6. What Problem It Solves

- Primary problem solved: delivering low-latency updates without per-event request overhead

Secondary benefit: better user experience, fewer empty requests, more natural live interaction patterns

- Systems impact: affects connection management, fan-out architecture, gateway behavior, load balancing, and state recovery

7. When to Rely on It

Use WebSockets when:

- the client and server both need to send messages frequently
- latency should be very low
- the experience is interactive, such as chat or collaborative editing
- you need full-duplex communication

Use SSE when:

- only the server needs to push updates
- the client mostly listens
- you want simpler HTTP-based streaming
- browser support and operational simplicity matter

Strong interviewer keywords for WebSockets:

- chat
- presence
- typing indicator
- collaborative editing
- live cursor movement
- bi-directional real-time interaction

Strong interviewer keywords for SSE:

- live feed
- notifications
- market ticker
- streaming logs
- progressive server output

8. When Not to Use It

Avoid WebSockets when:

- you only need one-way updates
- the product does not justify persistent connection complexity

Avoid SSE when:

- the client must also send frequent real-time messages back
- you need binary protocols or richer two-way session behavior
- you need very advanced connection semantics that the HTTP streaming model does not fit well

Better alternatives:

- polling or long polling for simpler or low-scale status updates
- gRPC or REST for normal request-response APIs
- webhooks for server-to-server notifications
- message queues for backend asynchronous processing rather than user-facing live delivery

9. Pros and Cons

| Pros | Cons |
|---|---|
| WebSockets enable real-time two-way communication | WebSockets add connection and session-management complexity |
| SSE is simpler than WebSockets for server-only push | SSE is one-way only |
| Both reduce repeated polling overhead | Persistent connections require careful scaling and reconnection handling |
| Both improve freshness and perceived responsiveness | Load balancers, proxies, and timeouts need special attention |
| SSE stays close to normal HTTP semantics | WebSockets can be harder to inspect, secure, and debug in some environments |

10. Trade-offs and Common Mistakes

Trade-offs

- Simplicity vs capability: SSE is simpler, WebSockets are more capable.
- One-way vs two-way: SSE is ideal for server-only updates, WebSockets are better when both sides talk actively.
- Operational ease vs interactive richness: SSE often works more naturally with HTTP infrastructure, while WebSockets unlock richer product behavior.
- Stateless HTTP mindset vs connection state: Push systems require thinking about reconnects, session continuity, and fan-out.

Common Mistakes

| Mistake | Why it is wrong | Better approach |
|---|---|--|
| Using WebSockets for simple one-way notifications | Adds unnecessary complexity | Use SSE if the client mostly listens |
| Using SSE for chat or collaboration | Client still needs another channel to talk back efficiently | Use WebSockets |
| Forgetting reconnect and resume behavior | Clients miss events after network drops | Track sequence IDs or last event IDs and replay safely |
| Storing too much state only in memory on one node | Reconnects and load balancer routing break continuity | Externalize session/event state where needed |
| Broadcasting every event to every connection | Fan-out cost becomes huge | Partition by user, room, topic, or shard |
| Ignoring heartbeat/idle timeouts | Connections silently die through proxies/load balancers | Use heartbeat, ping/pong, and timeout alignment |

11. Key Numbers

These are directionally useful interview numbers.

- Human-perceived "live" UX: often needs updates within about 100-1000 ms depending on the use case
- Heartbeat interval: often 15-30 seconds depending on infrastructure
- SSE reconnect delay: often 1-5 seconds, ideally with jitter
- Message fan-out concern: `connections x messages_per_second` quickly dominates server cost
- Connection memory: even a few KB per connection becomes significant at hundreds of thousands or millions of connections
- Gateway timeout alignment: app server, proxy, and load balancer timeouts must all allow long-lived connections

Simple fan-out math:

- 200,000 connected users x 2 events/sec = 400,000 event deliveries/sec

That is why push systems usually need dedicated fan-out infrastructure, topic partitioning, and careful capacity planning.

12. Failure Modes

Silent disconnects

Problem:

- Client thinks it is connected, but the socket or stream died through a proxy or network path.

User impact:

- Missed notifications, frozen presence, stale UI.

Mitigation:

- heartbeat
- ping/pong
- client-side idle detection
- reconnect with backoff

Reconnect storm

Problem:

- A deployment, region issue, or network flap causes many clients to reconnect at once.

User impact:

- sudden CPU, auth, and connection spikes

Mitigation:

- exponential backoff with jitter
- connection admission control
- staged restarts

Event loss after reconnect

Problem:

- Messages emitted while a client was offline are not replayed.

User impact:

- gaps in notifications or missed chat events

Mitigation:

- sequence numbers
- last event ID
- durable event log or message store

Hot fan-out nodes

Problem:

- one room, topic, or celebrity account causes concentrated broadcast traffic

User impact:

- uneven node load, latency spikes, dropped connections

Mitigation:

- shard by room/topic

• Separate chat channels

- pub/sub backbone for fan-out

13. Scenario

- Product / system: Team chat platform
- Why this concept fits: Users send messages, receive messages, see typing indicators, and update presence in real time.
- Recommended design: Use WebSockets for the interactive channel because both client and server need to speak continuously. Back the socket tier with a pub/sub system so messages fan out across multiple gateway nodes.
- What would go wrong without it: Polling would either feel stale or create huge request volume.

Second strong example:

- Product / system: Live incident dashboard
- Recommended design: Use SSE because the browser mostly listens for server-side status changes and alert updates.

14. Code Sample

Browser WebSocket client

```
const socket = new WebSocket("wss://api.example.com/v1/chat");

socket.onopen = () => {
  socket.send(JSON.stringify({ type: "join", roomId: "team-42" }));
};

socket.onmessage = (event) => {
  const message = JSON.parse(event.data);
  renderMessage(message);
};

socket.onclose = () => {
  setTimeout(connectAgainWithBackoff, 1000);
};
```

Browser SSE client

```
const eventSource = new EventSource("/v1/alerts/stream");

eventSource.onmessage = (event) => {
  const payload = JSON.parse(event.data);
  renderAlert(payload);
};
```

```

eventSource.onerror = () => {
  console.log("Stream interrupted, browser will retry");
};

```

Spring Boot SSE endpoint

```

@RestController
@RequestMapping("/v1/alerts")
public class AlertController {

    @GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<AlertEvent>> streamAlerts() {
        return alertService.stream()
            .map(event -> ServerSentEvent.<AlertEvent>builder()
                .id(event.id())
                .event("alert")
                .data(event)
                .build());
    }
}

```

Spring WebSocket handler sketch

```

@Component
public class ChatWebSocketHandler extends TextWebSocketHandler {

    @Override
    public void handleTextMessage(WebSocketSession session, TextMessage message) throws
    Exception {
        String payload = message.getPayload();
        // Parse the inbound chat message and publish it to the room/topic.
        chatService.handleInbound(session.getId(), payload);
    }
}

```

Key idea:

- SSE fits streaming outbound events.
- WebSockets fit interactive sessions where both sides send messages often.

15. Mini Program / Simulation

This small simulation compares delivery cost for polling vs push from a very high level.

```

def polling_requests(clients: int, seconds: int, interval: int) -> int:
    polls_per_client = seconds // interval
    return clients * polls_per_client

def push_deliveries(clients: int, events_per_second: int, seconds: int) -> int:
    return clients * events_per_second * seconds

```

```
def main() -> None:
    clients = 50_000
    seconds = 60
    polling_interval = 5
    events_per_second = 1

    total_polls = polling_requests(clients, seconds, polling_interval)
    total_push_events = push_deliveries(clients, events_per_second, seconds)

    print("Clients:", clients)
    print("Window:", seconds, "seconds")
    print("Polling requests:", total_polls)
    print("Push deliveries:", total_push_events)
    print("Interpretation: push avoids empty polling, but fan-out can still be enormous.")

if __name__ == "__main__":
    main()
```

What this demonstrates:

- push removes repeated empty requests
- push does not remove scale problems; it shifts them into connection management and fan-out
- choice depends on traffic shape and interaction style

16. Practical Question

You are designing a collaboration product with chat, presence, typing indicators, and a live notification panel. Would you use WebSockets or SSE for each feature, and how would you handle reconnects, missed events, and scaling across multiple gateway nodes?

17. Strong Answer

I would use both, based on interaction direction.

For chat, typing indicators, and presence, I would use WebSockets because the communication is truly bi-directional. Clients need to send messages, read receipts, and presence updates quickly, and the server needs to push new events back immediately. I would keep the WebSocket tier mostly responsible for connection handling and authentication, then use a pub/sub backbone behind it so events can be fanned out across many gateway nodes.

For a notification panel or incident feed where the browser mostly listens, I would prefer SSE because it is simpler and stays close to normal HTTP streaming semantics. That reduces complexity when I do not need a full two-way session.

In both cases, I would plan for dropped connections by using heartbeats, reconnect with exponential backoff and jitter, and include sequence IDs or last event IDs so clients can recover missed events after reconnect. The main trade-off is that push improves freshness, but it introduces persistent connection management, gateway timeout tuning, and fan-out scaling challenges.

18. Revision Notes

- One-line summary: WebSockets are best for two-way live interaction, while SSE is the simpler choice for one-way server-to-client streaming.
- Three keywords: push, duplex, fan-out
- One interview trap: choosing WebSockets everywhere even when SSE is enough
- One memory trick: WebSocket is a conversation, SSE is a broadcast

Topic 4: Choosing the Right Communication Model

Track: Communication Models & Real-Time Systems Scope: Chat -> WebSockets, Live feed -> SSE, Periodic dashboard -> Polling, Internal service calls -> gRPC, Public APIs -> REST

1. Intuition

Do not think of communication models as competitors where one technology wins everything.

Think of them like vehicles:

- REST is the reliable car for public roads
- gRPC is the fast train for internal service travel
- Polling is checking the bus stop again and again
- SSE is subscribing to a live station announcement
- WebSockets are opening a live call where both sides speak continuously

The real architect skill is not knowing the definitions. It is choosing the right model for the traffic shape, client type, and interaction pattern.

2. Definition

- Definition: Choosing between communication models means selecting the protocol and interaction style that best fit the system boundary, latency need, traffic pattern, and operational constraints.
 - Category: Architectural decision framework
 - Core idea: Pick the simplest model that satisfies the product and scale requirements, not the most fashionable one.
-

3. Why It Exists

Teams often make poor communication choices when they optimize for trend, not fit.

This topic exists because:

- a chat system and a public CRUD API do not have the same communication needs
- the wrong model creates avoidable latency, complexity, and cost
- interviewers want to see whether you can map requirements to architecture

Without this thinking:

- teams use WebSockets for everything
- or force everything through REST even when real-time interaction is required
- or over-poll and overload infrastructure

4. Reality

Real systems usually use multiple models together.

A single product might use:

- REST for mobile/web public APIs
- gRPC between backend services
- SSE for notifications or live feeds
- WebSockets for chat and presence
- Polling for status pages or as a fallback path

That is normal architecture maturity.

Boundary-first thinking

Use the system boundary to narrow choices:

- Public internet, broad client compatibility: REST usually wins first
- Internal service-to-service: gRPC is often a strong default
- Browser mostly listening: SSE is a strong candidate
- Browser/user actively sending and receiving live data: WebSockets are often best
- Periodic refresh with tolerable staleness: polling is often enough

5. How It Works

The selection flow should be deliberate.

1. Identify who the client is. Public browser/mobile app, internal service, admin console, or another backend.
2. Identify interaction direction. Request-response, server-to-client stream, or bi-directional conversation.

3. Identify business requirements. Minutes, seconds, sub-second, or near-instant.

4. Identify traffic shape. Periodic checks, sparse events, or continuous interaction.

5. Identify infrastructure constraints. Gateways, proxies, browser compatibility, mobile battery, debugging simplicity.

6. Choose the simplest model that satisfies the requirement.

7. Add fallbacks and failure handling.

Quick mapping

1. Public APIs -> REST

2. Internal low-latency service calls -> gRPC

3. Periodic dashboard/status -> Polling

4. Live one-way feed -> SSE

5. Interactive real-time session -> WebSockets

6. What Problem It Solves

- Primary problem solved: Selecting the right communication style for each use case instead of forcing one model everywhere
- Secondary benefits: Lower complexity, better latency, improved debuggability, and better cost efficiency
- Systems impact: Shapes API design, scaling model, fan-out architecture, client UX, and infrastructure choices

7. When to Rely on It

Use this decision framework whenever:

- you are designing a new API or real-time feature
- the interviewer asks "why this protocol and not that one?"
- a system needs multiple communication styles
- you need to justify trade-offs clearly

Specific defaults:

Use REST when:

- the API is public
- the client set is broad
- simplicity and compatibility matter most

Use gRPC when:

- services call each other internally

low latency and cost concerns matter

- generated clients and typed contracts are valuable

Use polling when:

- updates are periodic
- some staleness is acceptable
- the simplest solution is preferred

Use SSE when:

- the client mostly listens
- updates should arrive as they happen
- HTTP-friendly streaming is enough

Use WebSockets when:

- the interaction is real-time and bi-directional
- the client also needs to send frequent live events

8. When Not to Use It

Avoid REST when:

- the workload is extremely chatty internal RPC and efficiency matters more than internet compatibility

Avoid gRPC when:

- the consumer base is broad public web/mobile traffic and operational simplicity matters more than binary efficiency

Avoid polling when:

- the system needs live updates at scale
- most responses will be empty

Avoid SSE when:

- the client must also send frequent live messages

Avoid WebSockets when:

- the use case is simple one-way notification streaming
- the added connection complexity is not justified

Golden rule:

- do not choose the most powerful model by default
- choose the least complex model that still satisfies the requirement

9. Pros and Cons

| Model | Best for | Main strength | Main weakness |
|------------|--------------------------|-------------------------------|---|
| REST | Public APIs | Simple, debuggable, universal | Less efficient for chatty internal calls |
| gRPC | Internal service calls | Fast, typed, efficient | Less natural for public browser-facing APIs |
| Polling | Periodic checks | Easiest to build | Wasteful and stale |
| SSE | Live one-way feeds | Simple push over HTTP | One-way only |
| WebSockets | Bi-directional real-time | Interactive and low-latency | Harder to scale and operate |

10. Trade-offs and Common Mistakes

Trade-offs

- Simplicity vs capability: REST and polling are simpler; WebSockets are more capable.
- Compatibility vs efficiency: REST is broadly compatible; gRPC is more efficient internally.
- One-way vs two-way: SSE is enough for many live feeds; WebSockets are better for interactive sessions.
- Freshness vs infrastructure cost: Polling is easy but can waste requests; push models reduce empties but add connection management.
- Uniformity vs fitness: One protocol everywhere sounds clean, but mixed models are often the correct architecture.

Common Mistakes

| Mistake | Why it is wrong | Better approach |
|---|---|---|
| Using WebSockets for all real-time features | Overcomplicates one-way feeds and notifications | Use SSE where the client mostly listens |
| Using REST for internal high-frequency service hops | Adds avoidable overhead and looser contracts | Use gRPC for internal typed RPC |
| Using polling for chat | Either stale or too expensive | Use WebSockets |
| Using SSE for collaboration tools | Client still needs a second real-time channel back | Use WebSockets |
| Believing one system must use only one model | Real systems have different boundaries and traffic shapes | Mix models per use case |

11. Key Numbers

These are practical decision heuristics.

- Public API latency target: often p95 under 100-300 ms for common reads
- Internal gRPC hop: often a better fit when many service calls happen on the critical path
- Polling interval: often 5-30 seconds for dashboards or job status
- SSE or WebSocket live UX: often useful when updates should feel under about 1 second
- Fan-out warning: live push becomes costly when $\text{connections} \times \text{events_per_second}$ grows large
- Polling load warning: $\text{active_clients} / \text{polling_interval_seconds}$ gives rough request rate

Decision shorthand:

- slower and simpler -> polling or REST
- faster and interactive -> SSE or WebSockets
- faster and internal -> gRPC

12. Failure Modes

Wrong fit for the traffic pattern

Problem:

- The protocol works functionally but is a bad fit operationally.

User impact:

- stale data, high latency, unnecessary load, or needless system complexity

Mitigation:

- choose based on interaction pattern, not habit

Over-centralized protocol choice

Problem:

- Team mandates a single communication model for every boundary.

User impact:

- awkward client behavior, poor efficiency, and hard-to-justify trade-offs

Mitigation:

- allow mixed models by boundary and feature

Real-time without recovery design

Problem:

- Team chooses SSE or WebSockets but ignores reconnect, replay, and fan-out.

User impact:

| | |
|---|--|
| • missed events and fragile user experience | |
|---|--|

| | | |
|-------------|--|--|
| Mitigation: | | |
|-------------|--|--|

- sequence IDs, resume tokens, backoff, and pub/sub support

Polling overload

Problem:

- Dashboard or status clients poll too frequently.

User impact:

- high request volume and wasted compute

Mitigation:

- longer intervals, deltas, long polling, or switch to push

13. Scenario

Chat application

- Best choice: WebSockets
- Why: Users send messages, receive messages, show typing indicators, and update presence in both directions.
- Why not others: Polling is stale and wasteful, SSE is one-way, REST is not ideal for live interaction.

Live feed or notification stream

- Best choice: SSE
- Why: The server mostly needs to push updates to listening clients.
- Why not others: WebSockets would work, but usually add more complexity than needed.

Periodic dashboard

- Best choice: Polling
- Why: The UI can tolerate periodic refresh and simplicity matters more than perfect freshness.
- Why not others: Push infrastructure is often overkill for low-frequency refresh.

Internal service calls

- Best choice: gRPC
- Why: Low-latency typed service-to-service communication fits internal microservice boundaries well.
- Why not others: REST works, but gRPC is often more efficient and structured internally.

Public APIs

- Best choice: REST
- Why: Broad compatibility, easy debugging, and standard HTTP semantics make it a strong internet-

facing default.

- Why not others: gRPC is less natural for broad public use, and push models solve different problems.

14. Code Sample

Decision matrix in pseudocode

```
if client_is_public and interaction_is_request_response:
    use REST
elif client_is_internal_service and request_path_is_latency_sensitive:
    use gRPC
elif interaction_is_periodic_refresh and staleness_is_acceptable:
    use polling
elif interaction_is_server_to_client_only and updates_are_live:
    use SSE
elif interaction_is_bi_directional and real_time:
    use WebSockets
else:
    start with the simplest acceptable model
```

Example architecture mapping

```
communication_model_selection:
  public_mobile_api:
    model: REST
    reason: broad client compatibility
  internal_pricing_service_call:
    model: gRPC
    reason: typed low-latency RPC
  job_status_dashboard:
    model: polling
    reason: periodic refresh is enough
  incident_feed:
    model: SSE
    reason: one-way live updates
  team_chat:
    model: WebSocket
    reason: bi-directional real-time interaction
```

Java strategy enum sketch

```
public enum CommunicationModel {
    REST,
    GRPC,
    POLLING,
    SSE,
    WEBSOCKET
}

public class CommunicationAdvisor {
```

```

public CommunicationModel choose(boolean publicApi,
                                boolean internalService,
                                boolean periodicRefresh,
                                boolean oneWayLiveFeed,
                                boolean bidirectionalRealtime) {

    if (publicApi) return CommunicationModel.REST;
    if (internalService) return CommunicationModel.GRPC;
    if (periodicRefresh) return CommunicationModel.POLLING;
    if (oneWayLiveFeed) return CommunicationModel.SSE;
    if (bidirectionalRealtime) return CommunicationModel.WEBSOCKET;

    return CommunicationModel.REST;
}
}

```

15. Mini Program / Simulation

This small program maps a few use cases to the recommended communication model.

```

from dataclasses import dataclass

@dataclass
class UseCase:
    name: str
    public_api: bool = False
    internal_service: bool = False
    periodic_refresh: bool = False
    one_way_live_feed: bool = False
    bidirectional_realtime: bool = False

def choose_model(use_case: UseCase) -> str:
    if use_case.public_api:
        return "REST"
    if use_case.internal_service:
        return "gRPC"
    if use_case.periodic_refresh:
        return "Polling"
    if use_case.one_way_live_feed:
        return "SSE"
    if use_case.bidirectional_realtime:
        return "WebSockets"
    return "Start with the simplest acceptable model"

def main() -> None:
    use_cases = [
        UseCase(name="Public product API", public_api=True),
        UseCase(name="Inventory service to pricing service", internal_service=True),
        UseCase(name="Admin dashboard refresh", periodic_refresh=True),
        UseCase(name="Live stock ticker", one_way_live_feed=True),
        UseCase(name="Team chat", bidirectional_realtime=True),
    ]

```

```

for use_case in use_cases:
    print(f"{use_case.name}: {choose_model(use_case)}")

if __name__ == "__main__":
    main()

```

What this demonstrates:

- the choice is requirement-driven
- different features in one system can use different models
- strong design answers justify both selection and rejection

16. Practical Question

You are designing a collaboration SaaS platform with public REST APIs for external integrations, internal microservices for pricing and authorization, a live notification panel, a team chat feature, and a job-status dashboard. Which communication model would you choose for each feature, and why?

17. Strong Answer

I would choose communication models by boundary and interaction type, not force one model across the whole platform.

For public external APIs, I would use REST because it is broadly compatible, easy to debug, and a strong default for internet-facing consumers. For internal service-to-service calls on latency-sensitive paths, I would use gRPC because it gives typed contracts and lower overhead. For the job-status dashboard, I would start with polling because periodic refresh is enough and the operational simplicity is valuable.

For the live notification panel, I would prefer SSE because the browser mostly listens and I only need server-to-client streaming. For team chat, I would use WebSockets because it is a truly interactive, bi-directional real-time feature with typing indicators, presence, and message delivery in both directions.

The key trade-off is that richer protocols like WebSockets solve more problems but introduce more operational complexity. So I would keep each feature on the simplest model that satisfies its requirements and allow the overall system to use multiple communication models where appropriate.

18. Revision Notes

- One-line summary: Choose communication models by boundary, directionality, freshness, and complexity tolerance, not by trying to standardize everything on one protocol.
- Three keywords: boundary, direction, fit
- One interview trap: proposing one protocol for every feature in the system
- One memory trick: public -> REST, internal -> gRPC, periodic -> polling, one-way live -> SSE, two-

